项目 11 人体跟随运动

1、 整体课时安排

本项目是个综合应用项目，各子模块的课时安排如下。

项目名称	人体跟随运动	学时
模块名称	目标识别应用	2
	人体跟随应用	3
	系统调试扩展	3

2、 目标识别应用

2.1、 目标识别模型

我们在本项目中采用的目标识别模型是 SSD MobileNet V3 Large COCO，这是一个用于目标检测的深度学习模型，它结合了 SSD（Single Shot MultiBox Detector）算法和 MobileNet V3 作为其骨干网络。这个模型在 COCO 数据集上进行了预训练，能够检测多种不同的物体。

SSD 算法是一种单次检测模型，它通过在不同尺度的特征图上进行检测来识别图像中的物体。这种方法可以减少计算量，因为它避免了使用额外的对象提议步骤和后续的像素或特征重采样阶段。SSD 模型在训练和推理时都很简单，因为它将所有计算封装在一个单一的网络中。

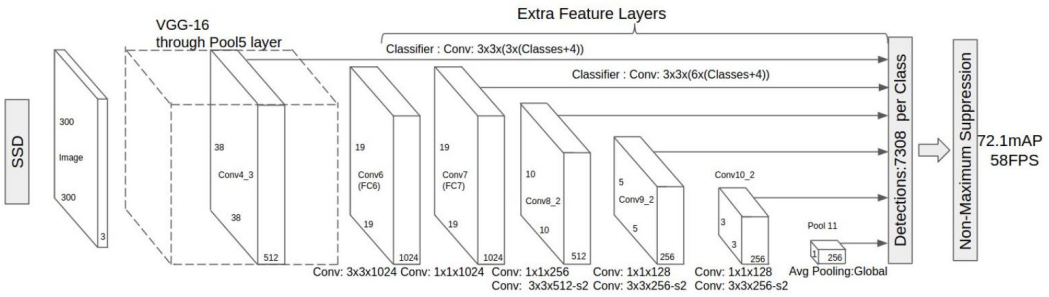


图 1-1 SSD 模型结构

MobileNet V3 是一个轻量级的深度神经网络，它通过使用深度可分离卷积（Depthwise Separable Convolution）来减少模型的参数数量和计算量。MobileNet V3 有两个版本：Large 和 Small，其中 Large 版本在保持较高准确率的同时，提供了更好的性能。

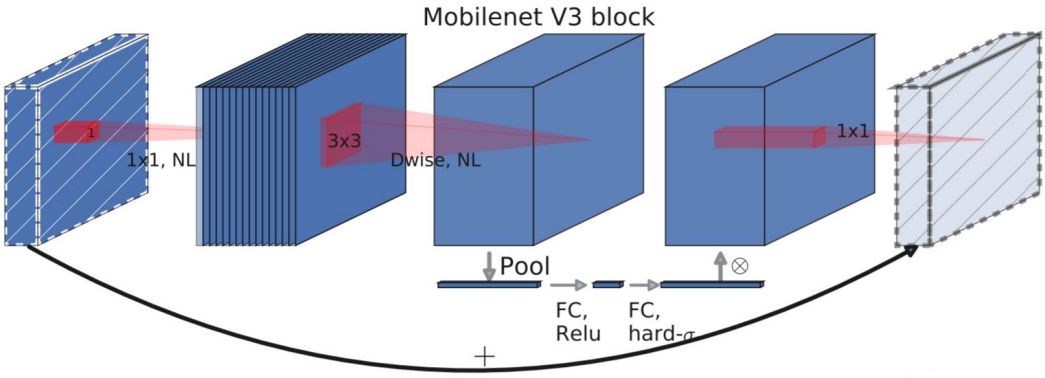


图 1-2 MobileNet V3 模型结构

结合 SSD 和 MobileNet V3 Large 的模型，即 SSD MobileNet V3 Large COCO，利用了 MobileNet V3 的轻量级特性和 SSD 的高效检测能力。这种组合使得模型在保持较高准确率的同时，能够在资源受限的设备上运行，例如在移动设备或嵌入式系统上进行实时目标检测。

此外，SSD MobileNet V3 Large COCO 模型在 COCO 数据集上进行了预训练，这意味着它已经学习了如何检测数据集中的 80 个类别，包括人、动物、日常物品等。这使得模型在实际应用中可以直接用于这些类别的检测，或者可以进行微调以适应新的类别或数据集。总的来说，SSD MobileNet V3 Large COCO 是一个高效的目标检测模型，适用于需要在保持高准确率的同时减少计算资源消耗的场景。

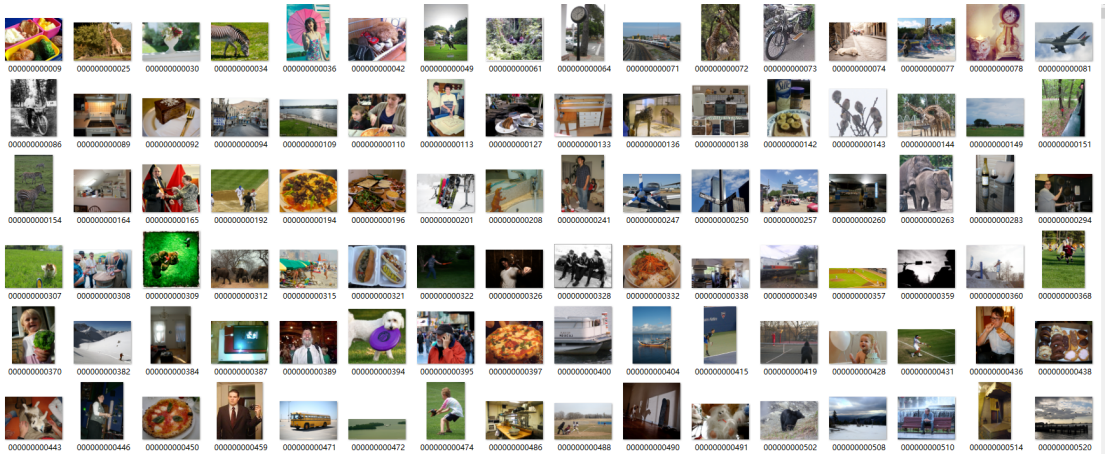


图 1-3 coco 数据集

2.2、 例程应用

2.2.1、 模型初始化

加载 coco 数据集物品名、模型训练权重和模型配置文件。

```
classNames = []
```

```
    classFile = 'coco.names'
    with open(classFile, 'rt') as f:
        classNames = f.read().rstrip('\n').split('\n')
    configPath = 'ssd_mobilenet_v3_large_coco_2020_01_14.pbtxt'
    weightsPath = 'frozen_inference_graph.pb'
    net = cv2.dnn_DetectionModel(weightsPath, configPath)
   model.setInputSize(320, 320)   #指定模型的输入大小
   model.setInputScale(1.0/127.5)   #缩放输入像素值
    model.setInputMean((127.5, 127.5, 127.5)) #设置输入预处理的平均减
去值
    model.setInputSwapRB(True) #交换输入图像中的红色和蓝色通道
```

2.2.2、 视频流初始化及阈值设置

```
    if len(sys.argv)>1:
            ChannelFactoryInitialize(0, sys.argv[1])
        else:
            ChannelFactoryInitialize(0)
        client = VideoClient()  # Create a video client
        client.SetTimeout(3.0)
```

```
client.Init()

code, data = client.GetImageSample()
```

2.2.3、 目标实时检测及绘制

使用置信度阈值为 0.5 的预训练模型对图像执行对象检测。它返回检测到的对象的类索引、置信度分数和边界框。

遍历检测到的对象，并在它们周围绘制边界框，并在输入图像上绘制相应的类标签和置信度分数，显示包含检测对象的输入图像。

```
code, data = client.GetImageSample()

# Convert to numpy image

image_data = np.frombuffer(bytes(data), dtype=np.uint8)

image = cv2.imdecode(image_data, cv2.IMREAD_COLOR)

classIds, confs, bbox = net.detect(image, confThreshold=thres)

if len(classIds) != 0:

    for classId, confidence, box in zip(classIds.flatten(), confs.flatten(), bbox): #flatten = so object

        cv2.rectangle(image, box, color=(0,255,0), thickness=3)

        cv2.putText(image, classNames[classId-1].upper(), (box[0]+10, box[1]+30), cv2.FONT_ITALIC, 1, (0,255,0), 2)
```

```
                cv2.putText(image,  str(round(confidence*100,2)),  (box[0]
+ 10, box[1] - 10), cv2.FONT_ITALIC, 1, (0, 255, 0), 2)
        # Display image
            cv2.imshow("front_camera", image)
```

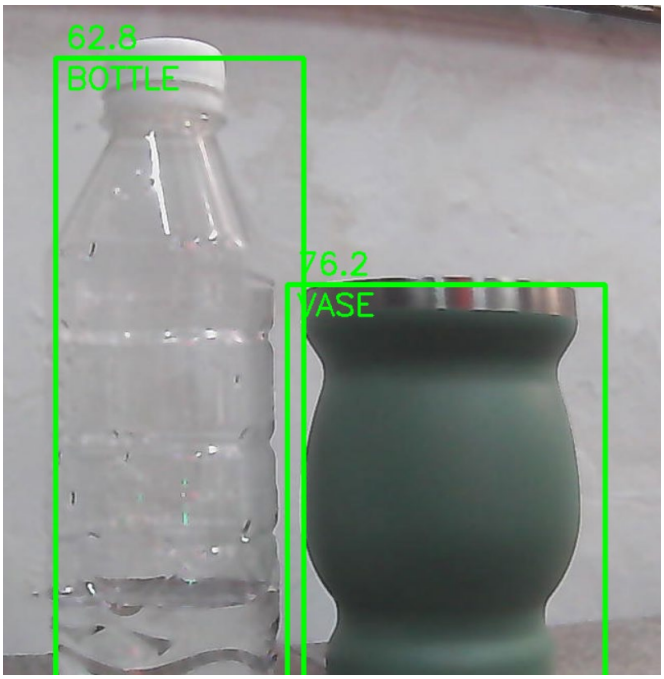


图 2-1 实时检测结果

3、 人体跟随应用

SSD MobileNet V3 Large COCO 模型的每个识别结果都会返回一个矩形框的元组数据 bbox，bbox 对应的信息如下。

bbox：[x_min (矩形框左上角 x 坐标), ymin（矩形框左上角 y 坐标）, width (矩形框宽度), height（矩形框高度）]。

我们将根据这个矩形的坐标和大小信息来设计目标跟随算法。

3.1、 目标跟随算法

Go2 机器人的默认视频区域是 1920*1080， 坐标系如下。

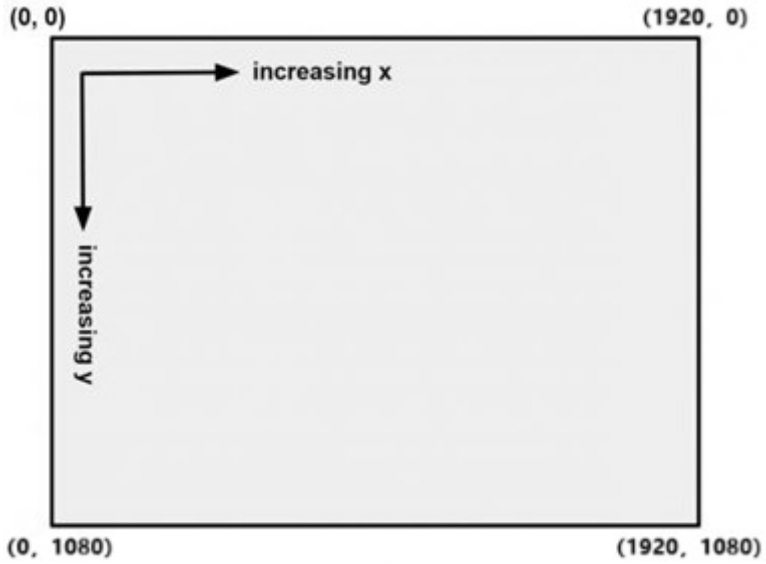


图 3-1 坐标系

3.1.1、 获得检测面积和坐标信息

因为我们当前需要追踪人体，所以先设置全局变量为“PERSON”标签。

```
#需要检测的特定物体
objectForDetect = "PERSON"
```

当目标物体为人体时，获取面积数据，并对获取的面积数据进行滤波处理，采样 5 次后，使用冒泡法排序后取中位数。

```
for classId, box in zip(classIds, bbox):
        #绘制识别标签
         draw_detection_area(classId, confs, bbox, image)
        #检测特定物体，计算面积和偏移量
         if(classNames[classId-1].upper()==objectForDetect):
            global index, filter_buf,FILTER_N
            #进行简单滤波算法
            if(index < FILTER_N):
                area_infor = list(box)
                filter_buf[index] = area_infor[2] * area_infor[3]
                index = index + 1
            else:
                index = 0
                #print(filter_buf)
                #进行冒泡排序
                filter_buf_sorted = sorted(filter_buf)
                area = filter_buf_sorted[(FILTER_N - 1) // 2]
                infors = list(box)
```

```
            loc_x = infors[0]

            loc_y = infors[1]

            for i in range(len(filter_buf)):

                filter_buf[i] = 0
```

根据面积大小获得远近信息，根据坐标大小获得偏移信息；然后给 Go2 机器人发送控制指令来追随目标。

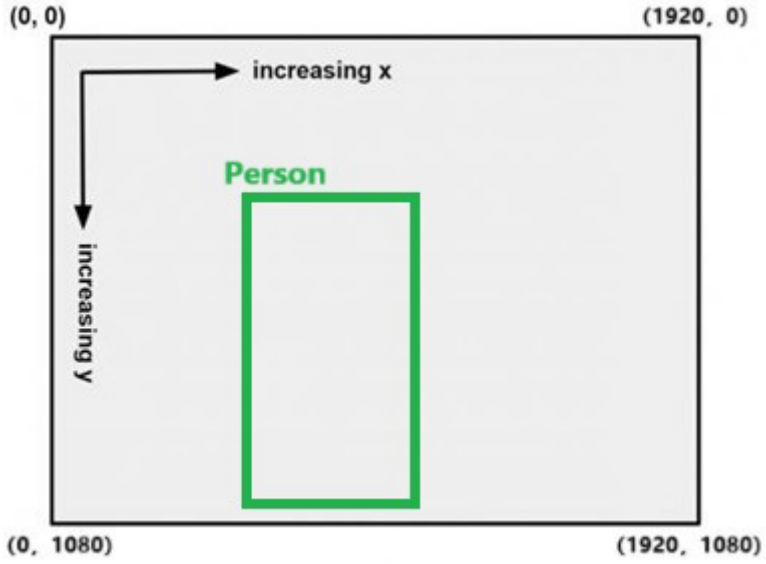


图 3-2 检测结果

```
def object_move_commond(loc_x, loc_y, area, pipe):

 if area < 600000:

 #发送前进命令

     pipe.send("Forward")
```

```
        if area > 1200000:
        #发送后退命令
            pipe.send("Backward")
        if loc_x > (1900/2+30):
        #发送左移指令
            pipe.send("Left")
        if loc_x <  (1900/2-30):
        #发送右移指令
            pipe.send("Right")
```

注意：以上运动控制的判断阈值需要根据具体检测目标和调试环境进行修改。

3.2、 机器人运动控制

3.2.1、 机器人运动控制类（SportModeTest）：

Forword、、Back、Left、Right 方法：分别用于控制机器人前进、后退、左移和右移的动作。调用 Move 函数实现前后左右运动，调用 StopMove 函数停止运动。

函数名	Move
函数原型	int32_t Move(float vx, float vy, float vyaw)
功能概述	移动。
参数	**vx:** 取值范围[-2.5~3.8] (m/s)； **vy:** 取值范围[-1.0~1.0] (m/s)； **vyaw:** 取值范围[-4~4] (rad/s)。
返回值	调用成功返回 0，否则返回相关错误码。
备注	控制移动速度。设定的速度为机体坐标系表示下的速度。

函数名	StopMove
函数原型	int32_t StopMove()
功能概述	停下当前动作，将绝大多数指令恢复成默认值。
参数	无
返回值	调用成功返回 0，否则返回相关错误码。
备注	停下当前的运动，并将 Go2 内部的运动参数恢复到默认值。

```
def Forward(self):
    elapsed_time = 1
    for i in range(int(elapsed_time / self.dt)):
        self.client.Move(0.1, 0, 0)  # vx, vy vyaw
        time.sleep(self.dt)
    self.client.StopMove()
def Backward(self):
    elapsed_time = 1
    for i in range(int(elapsed_time / self.dt)):
        self.client.Move(-0.1, 0, 0)  # vx, vy vyaw
        time.sleep(self.dt)
    self.client.StopMove()
```

```
        def Left(self):
            elapsed_time = 1
            for i in range(int(elapsed_time / self.dt)):
                self.client.Move(0, 0.1, 0)   # vx, vy vyaw
                time.sleep(self.dt)
            self.client.StopMove()
        def Right(self):
            elapsed_time = 1
            for i in range(int(elapsed_time / self.dt)):
                self.client.Move(0, -0.1, 0)   # vx, vy vyaw
                time.sleep(self.dt)
            self.client.StopMove()
```

3.2.2、 robot_control_process **函数**

接收目标识别线程发送的运动指令，并控制机器人执行相应的动作。

```
    def robot_control_process(pipe, network_interface, last_action_time_shared):
```

```
# 在子进程中重新初始化
ChannelFactoryInitialize(0,network_interface)
robot = SportModeTest()
while True:
    try:
        action = pipe.recv()  # 等待接收动作指令
        #print(f"Received gesture: {action}")
        now = datetime.now()
        time = now.strftime("%H:%M:%S")
        # 不需要冷却时间，立即执行动作
        if action == "Forward":
           print("robot is forward " + time)
           robot.Forward()
        elif action == "Backward":
            print("robot is backward" + time)
            robot.Backward()
        elif action == "Left":
            print("robot is left " + time)
```

```
                robot.Left()
            elif action == "Right":
                print("robot is right " + time)
                robot.Right()
```

3.3、 多线程应用

在主函数中采用了多进程来分离目标检测和机器人控制的任务，以提高程序的响应性和效率。我们使用 multiprocessing 库创建两个子进程，一个用于目标检测，一个用于机器人控制。这两个进程通过管道进行通信，通过共享变量 last_action_time_shared，用于在子进程间共享最后动作的时间。

```
    # 捕获系统信号（如 Ctrl+C）并终止进程
    signal.signal(signal.SIGINT, signal_handler)
    signal.signal(signal.SIGTERM, signal_handler)
    parent_conn, child_conn = mp.Pipe()
    network_interface = sys.argv[1]  # 获取网络接口
    # 创建一个共享的冷却时间变量
    last_action_time_shared = Value('d', time.time())
```

```
        # 创建手势检测进程和机器人控制进程
            p1 = mp.Process(target= object_detection_process,
args=(parent_conn, network_interface, last_action_time_shared))
                p2 = mp.Process(target=robot_control_process,
args=(child_conn, network_interface, last_action_time_shared))
        p1.start()
        p2.start()
```

4、 系统调试及扩展

以上代码和模块实现和搭建了人体跟随运动的大致框架，在此基础上，我们还可以给本系统加入调试模块及扩展功能。

4.1、 调试数据保存

将滤波后的面积数据存储进本地文件中，以便我们进行数据分析和确认控制阈值。

```
    def object_area_record(loc_x, loc_y, area):
        #将元组数据存储在文本文件中
        current_time = time.time()
        # 获取当前时间戳
```

```
            now = datetime.now()
            with open('data.txt', 'a') as file:
            # 将数据转换为字符串，并追加换行符
                line = "x:"+ str(loc_x) + "/y:" + str(loc_y) + "/area:" +
str(area) +   "/time:" + now.strftime("%H:%M:%S") + '\n'
               # 写入文件
                file.write(line)
```

```
x:709/y:11/area:873440/time:11:12:04
x:727/y:1/area:880139/time:11:12:04
x:735/y:0/area:1008321/time:11:12:05
x:711/y:5/area:996152/time:11:12:06
x:728/y:11/area:1181752/time:11:12:06
x:784/y:5/area:1159760/time:11:12:07
x:750/y:8/area:1116624/time:11:12:08
x:499/y:10/area:1113885/time:11:12:09
x:603/y:5/area:1178867/time:11:12:10
x:692/y:18/area:1176995/time:11:12:10
x:1129/y:12/area:1104480/time:11:12:11
x:779/y:0/area:607027/time:11:12:11
x:820/y:0/area:708800/time:11:12:12
x:1044/y:36/area:811512/time:11:12:13
```

图 4-1 调试数据输出

4.2、 上下追踪实现

我们还可以使用以下欧拉角控制函数，实现对目标物体的上下追踪。

函数名	Euler
函数原型	int32_t Euler(float roll, float pitch, float yaw)
功能概述	站立和行走时的姿态。
参数	**roll:** 取值范围 [-0.75~0.75] (rad)； **pitch:** 取值范围 [-0.75~0.75] (rad)； **yaw:** 取值范围 [-0.6~0.6] (rad)。
返回值	调用成功返回 0，否则返回相关错误码。
备注	设置 Go2 平衡站立或移动时的机体姿态角。欧拉角采用绕机体相对轴和 z-y-x 旋转顺序的表示方式。

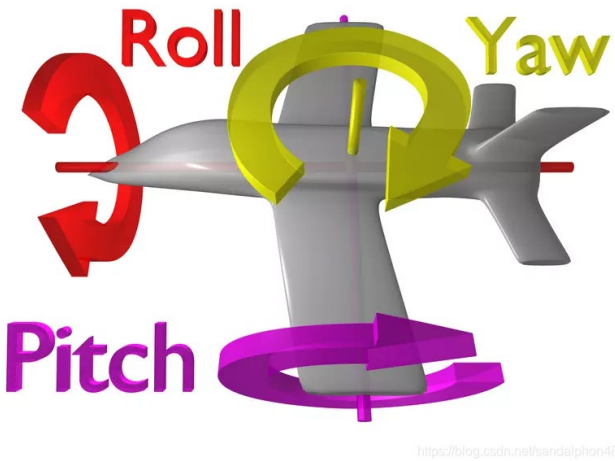


图 4-2 欧拉角控制

4.3、 多物体检测及追踪

我们还可以拓展代码和功能，实现多物体检测和追踪功能。

5、 项目总结

通过本项目的学习，我们学会了 SSD MobileNet V3 Large COCO 目标识别模型的使用，让 Go2 机器人追踪目标物体运动，并进行了相应的拓展，为最后进入大综合项目打下了坚实的基础。